

Name: Kanika Saini

SAP IP : 590034098

Program: Btech CSE B16

HDOC Day-24 —

Aim

To develop a **menu-driven C program** using a **switch statement** and **nested loops** to display different decreasing patterns. Each pattern is implemented using a **separate parameterless function with void return type**, without using arrays, as required in the given activity.

Algorithm

Main Program Algorithm

Step 1: Start the program.

Step 2: Declare an integer variable `choice`.

Step 3: Display the menu:

1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit

Step 4: Ask the user to enter a choice.

Step 5: Use a `switch` statement to process the choice.

- If `choice = 1`, call `pattern1()`.
- If `choice = 2`, call `pattern2()`.
- If `choice = 3`, call `pattern3()`.
- If `choice = 4`, call `pattern4()`.
- If `choice = 5`, terminate the program.

- Otherwise, display "Invalid choice!".

Step 6: After executing a pattern, display the menu again.

Step 7: Repeat until the user selects 5 (**Exit**).

Step 8: Stop.

Algorithms for Individual Functions

1. Decreasing Right-Angled Triangle

The PDF requires n rows of asterisks, with one fewer asterisk in every successive row. For $n = 4$, the rows contain 4, 3, 2, and 1 asterisks respectively.

Algorithm:

1. Start `pattern1()`.
2. Input the number of rows n .
3. Set outer loop i from n down to 1.
4. For each row, run inner loop j from 1 to i .
5. Print `*` during each inner-loop iteration.
6. Move to the next line after completing each row.
7. Return to the main program.

For $n = 4$:

```
* * * *
* * *
* *
*
```

2. Decreasing Natural-Number Pattern

The first row contains natural numbers from 1 to n , and the number of values decreases by one in each following row.

Algorithm:

1. Start `pattern2()`.
2. Input n .
3. Run outer loop i from n down to 1.
4. Run inner loop j from 1 to i .
5. Print the value of j .

6. Move to the next line after the inner loop.
7. Return to the main program.

For $n = 4$:

```
1 2 3 4
1 2 3
1 2
1
```

3. Decreasing Repeated-Number Pattern

The first row starts with n , printed n times. The value and the number of repetitions both decrease by one in each successive row.

Algorithm:

1. Start `pattern3()`.
2. Input n .
3. Run outer loop i from n down to 1.
4. Run inner loop j from 1 to i .
5. Print the value of i .
6. Move to the next line.
7. Repeat until $i = 1$.
8. Return to the main program.

For $n = 4$:

```
4 4 4 4
3 3 3
2 2
1
```

4. Reverse-Number Triangle

In every row, the numbers are printed in descending order. For $n = 4$, the first row is 4 3 2 1, followed by 3 2 1, and so on.

Algorithm:

1. Start `pattern4()`.
2. Input n .
3. Run outer loop i from n down to 1.
4. Run inner loop j from i down to 1.

5. Print j.
6. Move to the next line.
7. Repeat until all rows have been printed.
8. Return to the main program.

For n = 4:

```
4 3 2 1
3 2 1
2 1
1
```

Test-Case Table

Test Case	Menu Choice	Input n	Expected Output	Result
1	1	4	Decreasing * triangle with 4 rows	Pass
2	2	4	Natural numbers decreasing from 4 to 1 elements	Pass
3	3	4	Repeated numbers 4 4 4 4 to 1	Pass
4	4	4	Reverse-number triangle from 4 3 2 1 to 1	Pass
5	5	—	Program terminates	Pass
6	7	—	Invalid choice!	Pass

C Program

```
#include <stdio.h>

void pattern1();
void pattern2();
void pattern3();
void pattern4();

int main()
{
    int choice;

    do
    {
        printf("\n----- DECREASING PATTERNS MENU ----- \n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
```

```

scanf("%d", &choice);

switch(choice)
{
    case 1:
        pattern1();
        break;

    case 2:
        pattern2();
        break;

    case 3:
        pattern3();
        break;

    case 4:
        pattern4();
        break;

    case 5:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1 to 5.\n");
}

} while(choice != 5);

return 0;
}

/* Decreasing right-angled triangle */
void pattern1()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--)
    {
        for(j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

/* Decreasing natural-number pattern */
void pattern2()
{
    int n, i, j;

    printf("Enter number of rows: ");

```

```

scanf("%d", &n);

for(i = n; i >= 1; i--)
{
    for(j = 1; j <= i; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}

/* Decreasing repeated-number pattern */
void pattern3()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

/* Reverse-number triangle */
void pattern4()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--)
    {
        for(j = i; j >= 1; j--)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

Sample Execution

Test case1-

```
(kali㉿kali)-[~/Desktop]
$ cc day24.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

—— DECREASING PATTERNS MENU ——
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 4
* * *
* *
*
*
```

Test case 2-

```
—— DECREASING PATTERNS MENU ——
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
Enter number of rows: 4
1 2 3 4
1 2 3
1 2
1
```

Test case 3-

```
—— DECREASING PATTERNS MENU ——
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 3
Enter number of rows: 4
4 4 4 4
3 3 3
2 2
1
```

Test case 4-

```
—— DECREASING PATTERNS MENU ——
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 4
Enter number of rows: 4
4 3 2 1
3 2 1
2 1
1
```

Test case 5-

```
—— DECREASING PATTERNS MENU ——  
1. Decreasing right-angled triangle  
2. Decreasing natural-number pattern  
3. Decreasing repeated-number pattern  
4. Reverse-number triangle  
5. Exit  
Enter your choice: 5  
Exiting program...
```

Test case 6-

```
—— DECREASING PATTERNS MENU ——  
1. Decreasing right-angled triangle  
2. Decreasing natural-number pattern  
3. Decreasing repeated-number pattern  
4. Reverse-number triangle  
5. Exit  
Enter your choice: 7  
Invalid choice! Please enter 1 to 5.
```

Test-Case Evaluation

The program produces the required four decreasing patterns using nested loops without arrays. Each pattern is handled through a separate parameterless `void` function, satisfying the requirements given in the Day-25 activity.

Result: All prepared test cases produce the expected output, so the program is **successfully tested and verified**.